

Documentación Técnica del Proyecto

Monitor de Calidad del Aire y Datos Ambientales de América Latina — Materia Gestión de la Información, UTP.

Este documento explica, con fines de sustentación: (1) qué hace cada componente del código, (2) cómo se integran entre sí, (3) por qué el modelo estrella está diseñado así, y (4) qué representa cada página y gráfica del dashboard de Power BI.

1. Explicación del código

1.1 `config.py` — Configuración central

Es el punto de partida: casi todos los demás módulos importan algo de aquí.

En términos generales, esta es la única parte del proyecto donde se guardan datos que muchos otros archivos necesitan usar, como la lista de ciudades a monitorear o los colores que representan cada nivel de contaminación. La idea es simple: en vez de escribir esa información suelta y repetida en cada archivo (lo cual sería un problema si algún día hay que cambiarla), se centraliza aquí una sola vez, y el resto del sistema simplemente la consulta cuando la necesita.

- Carga `WAQI_TOKEN` y `GROQ_API_KEY` desde un archivo `.env` usando **python-dotenv**, para no dejar claves escritas directamente en el código.
- Define `CIUDADES`: un diccionario con las ~27 ciudades monitoreadas (latitud, longitud, id de estación WAQI).
- Define `PAIS_POR_CIUADAD`: mapea cada ciudad a su país — es lo que alimenta la columna `pais` de `dim_ciudad` en Power BI.
- Usa **pathlib** para definir y crear automáticamente las carpetas `data/`, `data/raw/`, `data/processed/`.
- Define `UMBRALES_AQI`: los rangos oficiales EPA (Bueno, Moderado, Dañino...) con su color y nivel de riesgo.
- `get_categoria_aqi(aqi)`: dado un valor numérico de AQI, busca en qué rango cae y devuelve su categoría, color y riesgo.
- `calcular_aqi_efectivo(aqi_reportado, pm25, pm10)`: calcula sub-índices AQI a partir de las concentraciones de PM2.5 y PM10 (siguiendo los puntos de corte EPA) y devuelve el **máximo** entre el AQI reportado por WAQI y esos sub-índices — esto evita que el sistema subestime la contaminación cuando WAQI reporta un valor desactualizado.
- Define constantes del modelo LLM: `GROQ_MODELO` (`llama-3.1-8b-instant`) y `GROQ_MAX_TOKENS`.

1.2 `pipeline/` — Ingesta, fusión y limpieza de datos

De forma general, esta carpeta agrupa todo lo relacionado con **obtener y preparar los datos** antes de que cualquier análisis pueda usarlos: se conecta a fuentes externas de internet, junta la información de calidad del aire con la de clima, limpia lo que venga incompleto o mal formado, y deja todo guardado en un formato ordenado y listo para usar. Es, en esencia, la fase de "recolección y limpieza" de todo el proyecto.

`ingesta_waqi.py`

En pocas palabras: este archivo se encarga de ir a buscar, por internet, los datos actuales de calidad del aire de cada ciudad.

- Consulta la API pública de WAQI (<https://api.waqi.info/feed/...>) usando la librería **requests**.
- Usa **ThreadPoolExecutor** (hasta 8 workers) para consultar las ~27 ciudades **en paralelo** en vez de una por una, reduciendo el tiempo total de la ingesta.
- Extrae AQI, PM2.5, PM10, CO, O3, NO2, SO2, temperatura y humedad de la respuesta JSON de cada ciudad.
- Normaliza todo en un `pandas.DataFrame` , una fila por ciudad, y guarda el crudo en `data/raw/waqi_*.json` .

`ingesta_clima.py`

De forma similar al anterior, este archivo busca los datos climáticos (temperatura, viento, lluvia, etc.) de cada ciudad, pero consultando una fuente distinta.

- Mismo patrón que el anterior, pero contra la API de Open-Meteo (sin necesitar clave).
- También usa **ThreadPoolExecutor** para paralelizar las consultas por ciudad.
- Extrae temperatura, temperatura aparente, humedad, viento, precipitación y código de clima (WMO), buscando la lectura horaria más cercana a "ahora".

`preprocesar.py`

Este es el archivo que toma los datos crudos recién descargados (que suelen venir incompletos, desordenados o con formatos distintos entre ambas fuentes) y los transforma en un conjunto de datos limpio, consistente y con información adicional calculada, listo para alimentar tanto los modelos de Machine Learning como el dashboard.

- `fusionar(df_waqi, df_clima)` : usa `pandas.merge` (tipo `outer`) para combinar ambas fuentes por la columna `ciudad` , evitando duplicar columnas que aparecen en ambos lados.
- `preprocesar(df)` :
- Convierte `timestamp` a datetime UTC con **pandas**.
- Elimina filas con más del 50% de valores nulos.
- **Interpola** linealmente los valores numéricos faltantes (excepto `codigo_clima` , que se rellena con el valor válido más cercano en el tiempo, porque es un código categórico, no una magnitud continua — interpolarlo generaría códigos WMO inexistentes).

- Genera features temporales: `hora_del_dia`, `dia_semana`, `es_fin_de_semana`, `mes`.
- Llama a `calcular_aqi_efectivo()` de `config.py` para obtener el AQI corregido.
- Llama a `get_categoria_aqi()` para derivar `categoria_aqi`, `color_aqi` y `riesgo_salud`.
- `guardar_procesado(df)`: guarda el resultado en `data/processed/datos.csv` (con deduplicación por `ciudad` + `timestamp` usando `drop_duplicates`) y en una base **SQLite** (tabla `lecturas`) usando el módulo estándar `sqlite3`.

`actualizar.py` — Orquestador

Este archivo es el que amarra todo el proceso anterior en una sola ejecución: en vez de tener que correr manualmente cada paso por separado, este orquestador los ejecuta uno tras otro en el orden correcto, y además puede repetir todo el proceso automáticamente cada hora si se necesita mantener los datos actualizados sin intervención humana.

- `ejecutar_pipeline()`: ejecuta en secuencia ingesta WAQI → ingesta clima → fusión → preprocesamiento → guardado → **predicción PM2.5 24h de todas las ciudades** → **resumen diario con LLM**.
- Cada paso se ejecuta a través de `_paso()`, que registra tiempo de inicio/fin y **atrapa errores por paso** (si una ciudad o fuente falla, no tumba el resto del pipeline).
- Usa la librería **schedule** para el modo programado (corre cada hora) y **argparse** para el flag `--now` (ejecución única).

1.3 `models/` — Machine Learning

Esta carpeta reúne los dos modelos de Machine Learning del proyecto. En términos generales, uno se encarga de **clasificar** qué tan buena o mala es la calidad del aire en una lectura puntual, y el otro de **predecir** cómo va a evolucionar la contaminación en las próximas horas — son dos preguntas distintas, y por eso se resuelven con dos técnicas distintas.

`clasificador.py`

En términos simples, este modelo aprende de los datos históricos a reconocer qué combinación de factores (nivel de contaminantes, hora, clima) corresponde a cada categoría de calidad del aire, para luego poder clasificar una lectura nueva sin necesidad de aplicar manualmente las reglas de la EPA.

- Técnica: **clasificación** con `RandomForestClassifier` de **scikit-learn**.
- Arma un `Pipeline` de sklearn: `StandardScaler` (normaliza variables) + `RandomForestClassifier` con `class_weight='balanced'` (para no ignorar categorías con pocas muestras, como "Peligroso").
- Usa `GridSearchCV` con `KFold` (validación cruzada) para probar combinaciones de `n_estimators` y `max_depth`, optimizando la métrica `balanced_accuracy`.
- Features usadas: `pm25`, `pm10`, `co`, `o3`, `temperatura`, `humedad`, `hora_del_dia`, `dia_semana`. Target: `categoria_aqi`.
- Serializa el modelo entrenado con **joblib** en `data/processed/clasificador.pkl`.

- `predecir(datos_dict)` y `get_feature_importance()` : funciones auxiliares para predecir una categoría puntual y para ver qué variables pesaron más en el modelo.

`prediccion.py`

En términos simples, este modelo mira cómo se ha comportado el PM2.5 de una ciudad en el pasado reciente y, a partir de ese patrón, estima cómo se va a comportar en las próximas 24 horas.

- Técnica: **regresión** con `LinearRegression` de scikit-learn.
- `_cargar_ciudad(ciudad)` : filtra el histórico de una ciudad desde `datos.csv` .
- `_predecir_lineal(df)` : entrena la regresión con `hora_del_dia` y `dia_semana` como features, predice PM2.5 para las próximas 24 horas, y calcula un margen de confianza (`lower` / `upper`) usando la desviación estándar de los residuales (con **numpy**).
- `entrenar_y_predecir(ciudad)` : función principal, guarda el resultado en `data/processed/predicciones_{ciudad}.csv` .

1.4 11m/ — Integración de LLM (Groq / Llama 3.1)

Esta carpeta contiene todo lo relacionado con el uso de un modelo de lenguaje (LLM) para generar texto en español a partir de los datos del proyecto — ya sea de forma automática (un resumen diario) o interactiva (respondiendo preguntas del usuario).

`resumenes.py`

Automatiza la redacción de un resumen diario del estado del aire en la región, sin que nadie tenga que escribirlo a mano.

- `_get_client()` : crea el cliente de **Groq** de forma perezosa (solo si hay `GROQ_API_KEY` configurada).
- `generar_resumen_diario(df)` : calcula métricas globales (ciudad con mayor/menor AQI, promedio de PM2.5, ciudades sobre AQI 100) y arma un **prompt** pidiéndole al LLM que redacte un resumen de máximo 150 palabras con esos datos.
- `generar_alerta(ciudad, aqi)` : prompt más corto para una alerta puntual por ciudad.
- `_limpiar_texto()` : colapsa saltos de línea a espacios, para que el texto generado no rompa el formato CSV al exportarlo.
- `guardar_resumen_diario(df, processed_dir)` : persiste el resumen en `data/processed/resumenes_diarios.csv` (reemplazando el del mismo día si ya existía), que es lo que alimenta `fact_resumenes` en Power BI.
- Todas las llamadas están en `try/except` : si Groq falla o no hay clave, se devuelve un texto de respaldo generado localmente en vez de romper el pipeline.

`consultas.py`

A diferencia del anterior, aquí el usuario tiene el control: en vez de recibir siempre el mismo tipo de resumen, puede preguntar lo que quiera sobre el estado actual de los datos y recibir una respuesta redactada en lenguaje natural.

- Es la funcionalidad de **preguntas en lenguaje natural** (la que le agrega interactividad al LLM, más allá del resumen automático).
- `_construir_contexto(df)` : arma un bloque de texto con la última lectura de cada ciudad (AQI, categoría, PM2.5, PM10, temperatura, humedad, viento).
- `responder_pregunta(pregunta, historial)` : arma un mensaje de sistema que incluye ese contexto + instrucción de "responde solo con estos datos, no inventes", y se lo manda a Groq junto con la pregunta del usuario (y opcionalmente el historial de la conversación, para mantener contexto entre preguntas seguidas).
- Se usa desde `menu.py` (opción 4) para una sesión de preguntas y respuestas en la terminal.

1.5 `exportar_powerbi.py` — Generador del modelo estrella

En términos generales, este archivo no crea información nueva — toma todo lo que ya generaron el pipeline, los modelos y el LLM, y lo reorganiza en el formato específico de tablas relacionadas (modelo estrella) que Power BI necesita para poder construir el dashboard.

- Lee `datos.csv` y los CSV auxiliares (`predicciones_*.csv` , `resumenes_diarios.csv`) con **pandas**.
- `normalizar_ciudad()` : corrige variantes de nombre de ciudad (ej. "Panama City" → "Ciudad de Panamá") para que los `JOIN` funcionen en Power BI.
- Seis funciones `construir_*` , una por tabla del modelo estrella (ver sección 3).
- `exportar_a_carpeta()` : guarda cada tabla como CSV individual en `data/processed/modelo_estrella/` .
- `exportar_a_excel_si_disponible()` : si están instaladas **openpyxl/xlsxwriter**, también arma un Excel con una hoja por tabla.

1.6 `menu.py` — Punto de entrada único

Es la puerta de entrada pensada para que cualquier persona (no solo quien programó el proyecto) pueda operar todo el sistema sin tener que memorizar comandos de terminal — basta con elegir un número de un menú.

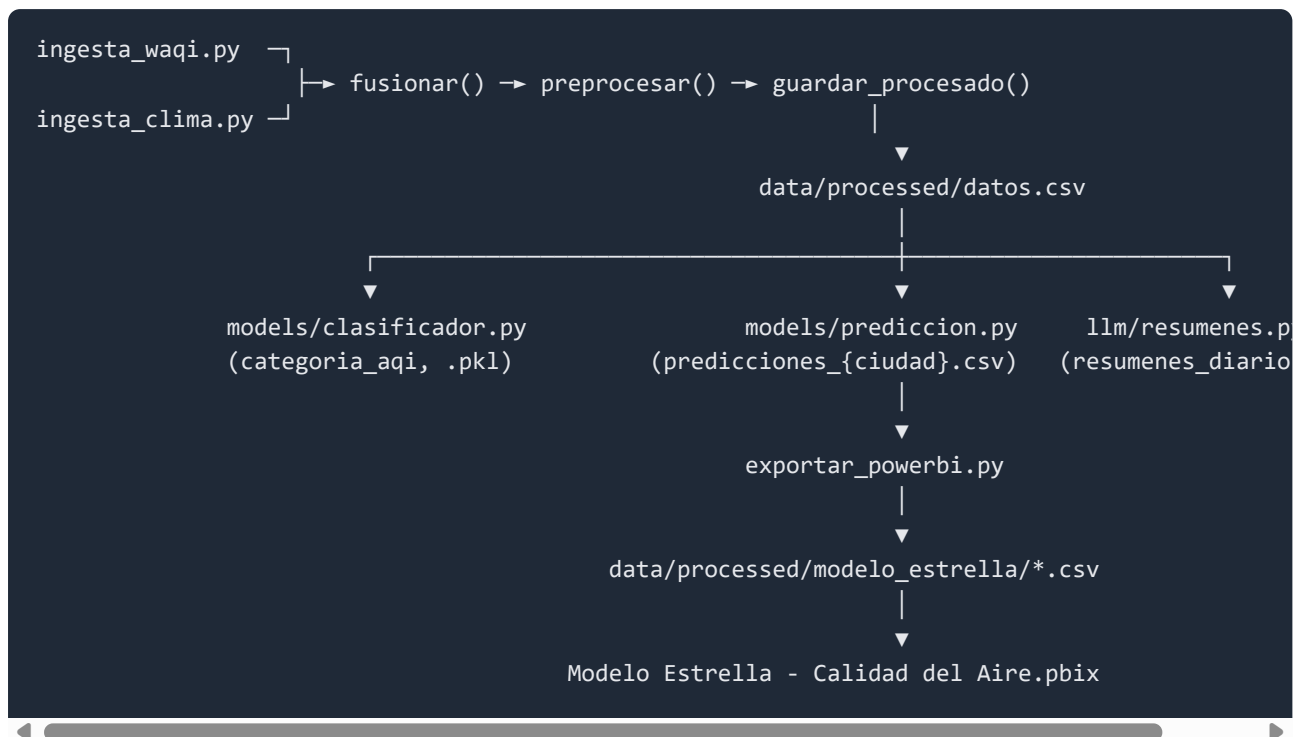
- Menú de terminal en bucle (`while True` + `input()`) que centraliza todas las operaciones: pipeline (con submenú para elegir modo único o programado), exportación a Power BI, entrenamiento del clasificador, y consultas al LLM.
- No contiene lógica propia de negocio — solo importa y llama a las funciones ya explicadas arriba.

2. Cómo se integran los componentes

El proyecto sigue un flujo **ETL clásico** (Extract → Transform → Load) con dos capas de análisis (ML y LLM) enganchadas al final, y una capa de presentación (Power BI) que consume el resultado.

En términos generales, ningún archivo del proyecto trabaja solo: cada uno recibe algo de un paso anterior y entrega su resultado al siguiente, como una cadena de producción. A continuación se

muestra ese recorrido completo, desde que se descarga el primer dato hasta que aparece en una gráfica de Power BI:



Cómo se conectan en la práctica:

- **config.py es el pegamento:** todos los módulos (`pipeline/` , `models/` , `llm/` , `exportar_powerbi.py`) importan de ahí las rutas de archivos, la lista de ciudades y los umbrales AQI — así hay una sola fuente de verdad, no valores repetidos en cada archivo.
- **pipeline/actualizar.py orquesta el ETL completo y lo conecta con ML y LLM:** después de guardar `datos.csv` , ese mismo archivo llama a `models/prediccion.py` (para las 27 ciudades) y a `llm/resumenes.py` , en ese orden — por eso una sola ejecución de `python -m pipeline.actualizar --now` deja todo listo para exportar.
- **exportar_powerbi.py es el puente entre Python y Power BI:** no genera datos nuevos, solo **reestructura** lo que ya existe (`datos.csv` , las predicciones, el resumen) al formato de modelo estrella que Power BI necesita.
- **llm/consultas.py y menu.py son la capa de interacción:** se conectan a los mismos datos (`datos.csv`) pero no participan del pipeline automático — se activan solo cuando el usuario los invoca manualmente desde el menú.
- **El clasificador (models/clasificador.py) es independiente del resto:** se entrena por separado (`python -m models.clasificador`) y no forma parte de la ejecución automática del pipeline; sus resultados (accuracy, importancia de features) se explican en la sustentación pero no se exportan a Power BI.

3. Modelo estrella: diseño y justificación

3.1 Por qué está diseñado así

Antes de entrar en el detalle técnico: un modelo estrella es simplemente una forma de organizar los datos en dos tipos de tablas para que Power BI pueda cruzarlos rápido y de forma clara — las tablas de **hechos** (los números que se miden, como una lectura de AQI) y las tablas de **dimensión** (los catálogos que le dan contexto a esos números, como el nombre de la ciudad o la categoría de riesgo). En vez de tener una sola tabla gigante con todo mezclado, se separan para que cada una tenga un propósito claro y las relaciones entre ellas sean simples de establecer.

El modelo tiene **3 tablas de hechos** (`fact_lecturas` , `fact_predicciones` , `fact_resumenes`) en vez de una sola. Esto no es un error de diseño — es una aplicación del principio de **separación de responsabilidades** en modelado dimensional (lo que en la técnica de Kimball se llama **esquema de constelación / galaxia de hechos**): varias tablas de hechos comparten dimensiones "conformadas" (`dim_ciudad` , `dim_fecha` , `dim_categoria_aqi`), pero cada una representa un **proceso de negocio distinto**, con su propio **grano** (nivel de detalle):

Tabla de hechos	Proceso que representa	Grano (una fila = ...)
<code>fact_lecturas</code>	Medición real de calidad del aire y clima	una lectura de una ciudad en un momento dado
<code>fact_predicciones</code>	Estimación del modelo de regresión	una hora futura pronosticada para una ciudad
<code>fact_resumenes</code>	Texto generado por el LLM	un resumen diario (a nivel región, no por ciudad)

Mezclar estas tres en una sola tabla violaría la regla de "un grano consistente por tabla de hechos": habría que rellenar con nulos las columnas que no aplican a cada tipo de fila (ej. `resumen_texto` no tiene sentido en una fila de predicción). Separarlas evita ese problema y hace que cada tabla sea más fácil de entender y de relacionar.

3.2 Tablas de hechos

Estas son las tablas que contienen los datos que realmente se miden o se generan:

- **`fact_lecturas`** : la tabla principal. Una fila por cada lectura real de una ciudad — `ciudad` , `timestamp` , `aqi` , `aqi_reportado` , `pm25` , `pm10` , `co` , `o3` , `no2` , `so2` , `temperatura` , `humedad` , `temperatura_aparente` , `viento_kmh` , `precipitacion_mm` , `codigo_clima` , `categoria_aqi` , `fecha` .
- **`fact_predicciones`** : una fila por cada hora pronosticada (24 por ciudad) — `ciudad` , `hora` , `pm25_predicho` , `lower` , `upper` .
- **`fact_resumenes`** : una fila por cada día — `fecha` , `timestamp_generacion` , `ciudad_max_aqi` , `ciudad_min_aqi` , `promedio_pm25` , `ciudades_sobre_100` , `resumen_texto` .

3.3 Tablas de dimensión

Estas son las tablas de catálogo que le dan contexto y significado a los datos de las tablas de hechos:

- **dim_ciudad** : catálogo maestro de ciudades — `ciudad` , `pais` , `latitud` , `longitud` , `waqi_id` . Se conecta a `fact_lecturas` y `fact_predicciones` .
- **dim_categoria_aqi** : catálogo de las 6 categorías EPA — `categoria_aqi` , `color_aqi` , `riesgo_salud` , `aqi_min` , `aqi_max` . Se conecta a `fact_lecturas` .
- **dim_tiempo** : calendario a nivel de **timestamp** (una fila por cada marca de tiempo distinta del dataset) — `hora` , `día` , `día_semana` , `nombre_día` , `mes` , `nombre_mes` , `año / year` , etc. Como su columna `fecha` se repite muchas veces (todas las horas del mismo día comparten fecha), **no es apta para relacionarse directamente** con las tablas de hechos.
- **dim_fecha** (tabla calculada en Power BI con DAX, no exportada por Python): resuelve el problema anterior. Usa `SUMMARIZE` sobre `dim_tiempo` para colapsar a **una fila por fecha única**, agregando los atributos de calendario con `MAX` . Esta es la tabla que realmente se relaciona con `fact_lecturas` y `fact_resumenes` por `fecha` , porque sí cumple la regla de valores únicos que exige una relación de Power BI.

3.4 Relaciones del modelo

Así es como todas las tablas terminan conectadas entre sí dentro de Power BI:

```
dim_ciudad[ciudad]      → fact_lecturas[ciudad]
dim_ciudad[ciudad]      → fact_predicciones[ciudad]
dim_categoria_aqi[categoria_aqi] → fact_lecturas[categoria_aqi]
dim_fecha[fecha]        → fact_lecturas[fecha]
dim_fecha[fecha]        → fact_resumenes[fecha]
dim_tiempo               → (fuente de dim_fecha, sin relación directa a los hechos)
```

Todas son relaciones "varios a uno" (`*:1`), con las tablas `dim_` del lado "uno" — la estructura estrella clásica.

4. Explicación del dashboard, página por página

El dashboard se organizó en **4 páginas temáticas** en vez de una sola sobrecargada, cada una con un propósito claro y sus propios slicers. A propósito, **los slicers NO están sincronizados entre páginas**: cada página es autosuficiente y muestra su propia historia completa, sin depender de un filtro que haya quedado seleccionado en otra página — importante si en la sustentación alguien salta directo a una página sin pasar por las demás.

En términos generales, cada página responde una pregunta distinta: la primera responde "¿cómo estamos ahora, en general?", la segunda "¿qué ciudades y patrones destacan?", la tercera "¿el clima tiene algo que ver?", y la cuarta "¿qué se espera que pase después?".

4.1 Página 1 — "Resumen General"

Es la portada: una foto instantánea del estado actual de la región. Pensada para que, con un solo vistazo (sin filtrar nada ni interactuar), cualquier persona entienda qué tan grave está la situación general en este momento.

Qué representa cada visual individualmente: - **AQI Promedio** (tarjeta): promedio de AQI de todas las lecturas visibles. - **AQI Máximo** (medidor/gauge): el peor AQI registrado, con una escala visual de 0 a 500 (el rango completo EPA) para dar noción de qué tan grave es ese máximo en contexto. - **Ciudades Monitoreadas**: cuántas ciudades tienen datos. - **Ciudades en Riesgo (AQI>100)**: cuántas superan el umbral de riesgo para grupos sensibles. - **% Ciudades en Riesgo**: la proporción anterior expresada en porcentaje — da una lectura más "de impacto" que el número absoluto. - **Mapa** ("AQI Promedio por latitud, longitud y ciudad"): un círculo por ciudad, con tamaño/color proporcional a su AQI promedio — permite ver de un vistazo la distribución **geográfica** de la contaminación en América Latina. - **Resumen del Día**: texto generado por el LLM que le da contexto narrativo a los números — es la única pieza del dashboard con lenguaje natural en vez de cifras.

Cómo funcionan en conjunto con los slicers: el slicer de **País** y el de **Fecha** filtran las 5 tarjetas y el mapa simultáneamente — si seleccionas "Chile" y un rango de fechas, tanto los KPIs como los círculos del mapa se recalculan solo con esos datos. El **Resumen del Día** es la excepción intencional: usa una medida (`CALCULATE` + `ALL`) que **ignora los slicers** a propósito, porque siempre debe mostrar el resumen más reciente generado por el LLM, sin importar qué esté filtrado en el resto de la página — es una "verdad fija" del día, no un dato explorable.

4.2 Página 2 — "Calidad del Aire"

La página de análisis profundo: compara ciudades, categorías y patrones temporales. A diferencia de la portada, aquí se espera que el usuario interactúe activamente con los filtros para ir descubriendo detalles específicos, en vez de solo observar un resumen fijo.

Qué representa cada visual individualmente: - **PM2.5 Promedio / PM10 Promedio** (tarjetas): los dos contaminantes más usados como referencia de calidad del aire. - **Lecturas Totales**: volumen de datos procesados — da noción de qué tan robusto es el análisis. - **AQI Promedio por país** (columnas, coloreadas por severidad): compara qué países tienen, en promedio, peor calidad del aire — Guatemala aparece como el más crítico en la captura, seguido de una franja intermedia en amarillo y varios países "buenos" en verde. - **Lecturas por Categoría AQI** (dona): qué proporción de todas las lecturas cae en cada categoría EPA (Bueno, Moderado, Dañino...) — muestra la distribución general de severidad, no solo el promedio. - **AQI Promedio por día de la semana** (columnas, ordenadas Monday→Sunday): busca patrones semanales (¿hay días con más tráfico/industria que otros?). - **PM2.5 Promedio por Fecha y País** (líneas): la evolución temporal del contaminante principal, una línea por país — permite ver tendencias y picos puntuales (como el pico rojo visible a mediados de junio). - **Tabla "Alertas por Ciudad"**: la **última lectura real** de cada ciudad (no un promedio del período), ordenada de peor a mejor AQI y coloreada por riesgo — es la vista de "estado actual" ciudad por ciudad, pensada para identificar rápidamente dónde hay que poner atención ahora mismo.

Cómo funcionan en conjunto con los slicers: los 3 slicers (**País**, **Ciudad**, **Fecha**) filtran los 4 gráficos y la tabla a la vez — es la página más interactiva del dashboard, pensada para "investigar": por ejemplo,

filtrar a un solo país y un rango de fechas para ver si su AQI por día de semana o su tendencia de PM2.5 cambia.

4.3 Página 3 — "Datos Ambientales"

La página de clima, con un propósito adicional: **conectar el clima con la calidad del aire**, no mostrarlo aislado. La pregunta de fondo que responde esta página es si el clima de una ciudad ayuda a explicar por qué su aire está más o menos contaminado.

Qué representa cada visual individualmente: - **Temperatura Promedio / Máxima / Mínima, Humedad Promedio, Viento Promedio, Precipitación Promedio** (6 tarjetas): resumen estadístico de las variables climáticas. - **Temperatura Promedio en el Tiempo** (línea, por país): evolución temporal, igual que la de PM2.5 en la página anterior pero para clima. - **Temperatura Promedio por País / Viento Promedio por País** (columnas): comparación entre países. - **Temperatura vs. Calidad del Aire por Ciudad** (dispersión, tamaño = PM2.5): cada punto es una ciudad; busca si las ciudades más calurosas tienden a tener peor AQI. - **Humedad vs. PM2.5 por Ciudad** (dispersión): mismo concepto, pero cruzando humedad con PM2.5 directamente — estos dos gráficos son los que le dan **propósito analítico** a la página, más allá de solo describir el clima.

Cómo funcionan en conjunto con los slicers: **País, Ciudad y Fecha** filtran todos los visuales de la página. Es especialmente útil combinarlos con los gráficos de dispersión: al filtrar a un solo país, se puede ver con más claridad si sus ciudades siguen el mismo patrón clima-contaminación o no.

4.4 Página 4 — "Predicciones"

La única página "hacia adelante" — no describe el presente, anticipa las próximas 24 horas. Está pensada para responder, de forma anticipada, si conviene tomar precauciones en una ciudad específica antes de que la contaminación realmente suba.

Qué representa cada visual individualmente: - **PM2.5 Predicho Promedio / Máximo** (tarjetas): resumen del pronóstico para la ciudad seleccionada. - **Ciudades con Pronóstico**: cuántas ciudades tienen un forecast calculado (control de cobertura del modelo). - **% Horas en Riesgo (Predicción) / Horas Pronosticadas en Riesgo**: de las 24 horas pronosticadas, cuántas/qué porcentaje supera el umbral de riesgo ($35.4 \mu\text{g}/\text{m}^3$ de PM2.5, equivalente a $\text{AQI} > 100$) — es el KPI más "de alerta" de esta página. - **Margen de Incertidumbre Promedio**: qué tan ancho es el intervalo de confianza del modelo de regresión lineal — un número alto significa que el modelo está menos seguro de su propia predicción. - **Pronóstico PM2.5 — Próximas 24h** (línea con 3 series: predicción central, banda inferior y banda superior): visualiza tanto el valor esperado como la incertidumbre del modelo a lo largo de las 24 horas siguientes.

Cómo funcionan en conjunto con los slicers: a diferencia de las otras páginas, el slicer de **Ciudad** aquí es de **selección única** (no múltiple) — deliberadamente, porque las predicciones son individuales por ciudad y mezclar varias en el mismo gráfico de línea sería confuso de leer. Al cambiar de ciudad, las 6 tarjetas y el gráfico de pronóstico se recalculan para esa ciudad específica.

5. Técnicas utilizadas y por qué

Esta sección resume las técnicas aplicadas en el proyecto y cómo cada una responde a un punto específico de lo solicitado. En pocas palabras, cada técnica se eligió no porque fuera la más avanzada posible, sino porque era la más adecuada para el tamaño y tipo de datos que maneja este proyecto — un criterio de decisión que también vale la pena explicar en la sustentación.

5.1 Machine Learning — Clasificación (Random Forest)

Se usó para predecir la **categoría AQI** (Bueno/Moderado/Daño/...) a partir de variables ambientales. Se eligió Random Forest porque: (a) captura relaciones **no lineales** entre contaminantes y categoría (el AQI cambia "a saltos" por umbrales, no proporcionalmente), (b) soporta `class_weight='balanced'` para manejar categorías con pocas muestras (ej. "Peligroso"), y (c) es robusto a outliers, comunes en datos de sensores ambientales reales. Resultado real obtenido: **94.4% de accuracy**, con PM2.5 y PM10 como las variables más determinantes (33% y 18% de importancia respectivamente).

5.2 Machine Learning — Regresión (Regresión Lineal)

Se usó para predecir PM2.5 a 24 horas. Se eligió un modelo lineal simple (en vez de algo más complejo como Prophet, que se probó y se descartó) porque con pocas variables predictoras (hora del día, día de la semana) y datos históricos limitados por ciudad, un modelo simple es más interpretable, más rápido de entrenar 27 veces en cada corrida del pipeline, y evita el sobreajuste que un modelo más complejo tendría con pocos datos.

5.3 Modelado dimensional — Esquema de constelación (Star Schema extendido)

Ver sección 3. Permite separar los 3 procesos de negocio (mediciones, predicciones, resúmenes) sin mezclar grados de detalle distintos en una sola tabla, cumpliendo la práctica estándar de modelado dimensional de Kimball.

5.4 LLM — Generación de resúmenes (prompt engineering)

`llm/resumenes.py` arma un prompt que combina métricas calculadas (no texto libre) con instrucciones específicas de formato y contenido, para que el LLM redacte un resumen consistente y basado en datos reales, no alucinado.

5.5 LLM — Consultas en lenguaje natural (context stuffing / prompt grounding)

`llm/consultas.py` aplica la misma idea de forma interactiva: en vez de dejar que el LLM "invente" respuestas sobre datos que no tiene, se le inyecta el estado actual real como contexto en cada pregunta, y se le instruye explícitamente a no responder más allá de esos datos. Es una versión simplificada de la técnica conocida como **RAG (Retrieval-Augmented Generation)** — aquí el "retrieval" es directo (todas las últimas lecturas caben en un solo prompt) en vez de requerir una base vectorial.

5.6 Ingesta paralela (conurrencia)

`ThreadPoolExecutor` en `ingesta_waqi.py` e `ingesta_clima.py` permite consultar las ~27 ciudades simultáneamente en vez de secuencialmente, reduciendo drásticamente el tiempo de cada corrida del pipeline (las APIs externas son el cuello de botella, no la CPU, por lo que hilos son apropiados aquí en vez de multiprocessing).

5.7 Resiliencia y manejo de errores por paso

Cada paso del pipeline (`_paso()` en `actualizar.py`) está aislado: si una ciudad falla en la ingesta, o una predicción no tiene suficientes datos, el resto del proceso continúa. Esto es importante porque el sistema depende de APIs externas (WAQI, Open-Meteo, Groq) que pueden fallar de forma parcial o intermitente.